

# Spatio-Temporal Traffic Forecasting and Dynamic Route Recommendation over Benchmark Traffic Graphs

Mittapalli Manikanta Naga Sai Lalitha, Pranav Yerra, Konjeti Gnana Venkata Naga Gopi Prasanth

Department of Computer Science and Engineering

Koneru Lakshmaiah Education Foundation (KLEF)

Guntur, Andhra Pradesh, India

Emails: {2300032492, 2300030761, 2300031235}@kluniversity.in

**Abstract**—Traffic forecasting and route selection are often implemented as separate computational problems, even though predicted traffic conditions directly influence route travel time. This paper presents a end-to-end prototype that connects learned spatio-temporal traffic-speed forecasting with deterministic graph-based route recommendation. The learned component is CPSTN-Draft5, a temporal-gated dense graph-convolutional architecture that processes twelve historical observations and forecasts twelve future speed steps at five-minute intervals, covering a nominal horizon from five to sixty minutes. The deterministic decision layer converts forecast speed into dynamic edge travel-time costs, generates candidate paths over the supplied benchmark topology, performs shortest-path search using a zero-heuristic A\* formulation, compares predicted travel time between the current and alternative paths, computes a historical-error-based reliability proxy, and applies configurable REROUTE/STAY policy thresholds.

On the supplied METR-LA test artifact, containing 6,844 test windows, CPSTN-Draft5 reports an MAE of 3.4454, RMSE of 6.8797, MAPE of 9.5662%, sMAPE of 7.9628%, WAPE of 5.9622%, and  $R^2$  of 0.74246. Horizon-wise evaluation reports RMSE increasing from 3.9419 at five minutes to 8.5677 at sixty minutes, while  $R^2$  decreases from 0.91543 to 0.60064. The complete prototype connects model inference, preprocessing, graph routing, decision logic, backend APIs, and frontend visualization.

A critical distinction is maintained throughout the study: the supplied benchmark graph contains connectivity information but does not provide a validated geographic road network with physical road lengths and geographic coordinates. Consequently, the routing layer uses deterministic proxy distances and demonstrates topology-based route computation rather than geographic turn-by-turn navigation. The work therefore evaluates the integration of traffic forecasting with deterministic benchmark-topology routing and explicitly does not claim experimentally validated live GPS navigation or live traffic routing.

**Index Terms**—traffic forecasting, spatio-temporal graph learning, graph neural networks, road networks, dynamic routing, travel time, A\*, shortest path, route recommendation, METR-LA, PEMS-BAY

## I. INTRODUCTION

Urban transportation systems exhibit strong temporal and spatial dependencies. Traffic speed at one observation location is affected not only by its recent local history but also by conditions propagating through connected portions of the transportation network. A forecasting model can therefore be

useful when it captures both temporal evolution and graph structure. However, forecasting alone does not directly answer an operational question faced by a route-selection system: given the predicted future traffic state, should the current path be retained or should an alternative path be considered?

This paper studies that question through a full-stack prototype centered on CPSTN-Draft5. The system separates the problem into two layers. The first layer is learned: CPSTN-Draft5 predicts future traffic speeds from historical traffic observations and the supplied benchmark graph. The second layer is deterministic: predicted speeds are converted into edge travel-time costs and supplied to graph-search and decision procedures. This separation is important because the routing algorithm is not learned by CPSTN, and CPSTN does not directly output a route. Instead, the forecast becomes an input to a conventional optimization and policy layer.

The distinction also makes the system easier to analyze. Forecast quality can be measured using standard regression metrics independently from route-selection behavior. The route layer can then be examined in terms of graph connectivity, cost construction, candidate generation, shortest-path search, and policy thresholds. Such separation avoids conflating predictive accuracy with downstream route quality.

The broader literature contains substantial work on road extraction, network topology, graph generation, map updating, and travel-time-aware routing. Batra *et al.* demonstrated that connectivity can be improved by jointly learning orientation and segmentation rather than relying only on pixel-wise road classification [1]. Wegner *et al.* similarly incorporated higher-order structural relationships into road-network extraction [4]. Belli and Kipf treated road-network extraction as image-conditioned graph generation [5]. Bastani and Madden introduced a dataset and evaluation framework for updating road maps from aerial imagery [2], while Sui *et al.* used intersection-change detection and directed road tracing for vector map updating [8]. Zhang and Couloigner addressed road change detection and map updating using feature matching, conflation, and spatio-temporal concepts [9]. More recent work by Duan *et al.* combines segmentation, graph optimization, and vector road-network generation for satellite imagery [7].

Traffic-aware road-network attributes provide another relevant connection. Van Etten’s city-scale road extraction work explicitly considered road speeds and travel times, demonstrating why travel-time-aware routing can differ from purely geometric route selection [3]. This principle is directly relevant to the present system: the learned output is speed, and the downstream graph layer transforms that speed into an estimated traversal cost.

Deep learning has also been applied extensively to high-resolution road extraction. Xu *et al.* proposed a deep-learning framework using contextual information for road extraction [6], while Zhu *et al.* developed a global context-aware and batch-independent network and examined transferability across datasets [10]. These studies are relevant to the broader road-network problem, but their principal task is image-to-road extraction rather than traffic forecasting.

Within the reviewed literature and the supplied project artifacts, the research position of this work is therefore deliberately narrow. The supplied studies address important components of the broader transportation-network problem, including connectivity, topology extraction, graph generation, map updating, and road-speed estimation. The supplied CPSTN work instead provides a traffic-speed forecasting implementation and integrates its predictions with a deterministic route-cost and decision layer. The paper does not claim that no previous study has ever combined these concepts. Rather, it reports the specific integration implemented and evaluated in the supplied project.

The principal contributions are as follows:

- 1) A reproducible CPSTN-Draft5 traffic forecasting configuration using twelve historical steps and twelve five-minute forecast steps.
- 2) A dynamic edge-cost mechanism that converts forecast speed and deterministic proxy distance into estimated traversal time.
- 3) A graph-based candidate route generation and shortest-path layer using the supplied benchmark topology.
- 4) A deterministic route decision mechanism that compares current and alternative predicted travel time and incorporates a historical-error reliability proxy.
- 5) A full-stack implementation connecting preprocessing, CPSTN inference, graph routing, decision logic, backend APIs, and frontend visualization.
- 6) An explicit evaluation framework that separates forecasting performance from topology-based routing behavior and clearly documents the limitations of the supplied benchmark graph.

The remainder of this paper is organized as follows. Section II reviews related work. Section III formulates the forecasting and routing problem mathematically. Section IV describes the CPSTN-centered methodology and software architecture. Section V presents the experimental setup. Section VI reports the supplied forecasting results and system-level routing validation. Section VII discusses the implications of forecast error for routing. Section VIII describes limitations and threats

to validity. Section IX concludes the paper and identifies future research directions.

## II. RELATED WORK

### A. Deep Learning for Road Extraction

Road extraction from remote-sensing imagery is commonly formulated as semantic segmentation, but isolated pixel accuracy does not necessarily produce a usable road network. Xu *et al.* proposed a DenseNet-based road-extraction architecture using local and global attention to improve contextual representation [6]. The work demonstrates the importance of multi-scale contextual features when roads contain complex intersections, shadows, occlusions, and visually ambiguous regions.

Zhu *et al.* subsequently introduced a global context-aware and batch-independent network for very-high-resolution satellite imagery [10]. Their analysis also considers cross-domain transfer, which is important because a model trained on one geographic or imaging distribution may not preserve the same extraction behavior elsewhere. Such transferability concerns are relevant to any system that ultimately intends to move from benchmark data toward geographic deployment.

These road-extraction studies differ from CPSTN in task definition. Their primary input is imagery, whereas CPSTN operates on historical traffic observations associated with a graph. Nevertheless, they motivate the broader representation of transportation infrastructure as structured networks rather than independent pixels.

### B. Connectivity and Topology-Aware Road Modeling

A road extraction system can produce visually plausible segments while still generating broken or disconnected networks. Batra *et al.* addressed connectivity by jointly learning orientation and segmentation and by explicitly considering connectivity refinement [1]. The underlying motivation is directly relevant to graph-based transportation systems: downstream routing depends on whether road segments form a connected structure.

Wegner *et al.* approached road-network extraction using a higher-order conditional random field that explicitly models road-network structure [4]. Such methods emphasize that roads have relationships extending beyond local pixels. Connectivity, smoothness, and network consistency can be critical even when local classification appears accurate.

The CPSTN prototype does not perform image-based topology extraction. Instead, it receives an already constructed benchmark graph. Consequently, the topology problem in this study is an input assumption rather than a learned component. This distinction prevents the routing layer from being presented as a solution to road extraction.

### C. Graph Generation and Map Updating

Graph representations provide a direct way to model roads as nodes and edges. Belli and Kipf proposed an image-conditioned graph-generation approach for road-network extraction and introduced a graph transformer formulation for generating road structures [5]. Their work is relevant because it demonstrates

a representation in which road structure is directly expressed as a graph.

Map updating addresses a related but different problem. Bastani and Madden introduced MUNO21, a dataset designed to study map updates from aerial imagery, including road additions, removals, and shifts while maintaining unchanged regions [2]. This is important because maintaining an existing road map differs from constructing an entirely new map.

Zhang and Couloigner presented a framework combining road extraction, change detection, feature matching, conflation, and spatio-temporal database concepts [9]. Sui *et al.* later considered intersection-change detection and directed road tracing for vector road-map updating [8]. Duan *et al.* presented a more recent automated satellite-image road-network extraction pipeline incorporating graph optimization and vector line-string generation [7].

These works establish a progression from image segmentation toward structured graph construction and map maintenance. The present CPSTN framework assumes the graph already exists and instead focuses on traffic-state prediction and route-cost adaptation.

#### D. Speed and Travel-Time-Aware Routing

Geometric shortest path is not equivalent to fastest path when travel speeds vary across edges. Van Etten’s city-scale road extraction work included road-speed and travel-time attributes and demonstrated their relevance for route planning [3]. This provides an important conceptual precedent for converting predicted speed into edge traversal time.

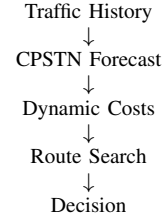
The present system adopts the same broad principle without claiming the same data source or geographic capability. For each benchmark edge, a deterministic proxy distance is combined with predicted speed. This produces a dynamic cost that can change when the predicted traffic state changes.

The routing layer therefore represents a conventional optimization stage after neural forecasting. It is not itself a graph neural network, and the  $A^*$  search is not trained by the CPSTN model.

#### E. Positioning of the Proposed Integration

The reviewed literature spans several layers of a larger transportation-information pipeline. Road-extraction studies address how roads can be recovered from imagery. Topology-aware methods address network connectivity. Graph-generation studies address direct graph representations. Map-update studies address network freshness. Speed-aware road-network work addresses travel-time attributes.

The supplied CPSTN implementation addresses the forecasting layer over traffic graphs. The prototype connects that layer to deterministic graph routing. The resulting system can therefore be viewed as an integration architecture:



The distinction is important for the novelty claim. The contribution is not the invention of graph routing,  $A^*$ , road extraction, or traffic forecasting individually. The contribution is the implementation and evaluation of their particular integration around the supplied CPSTN-Draft5 model, together with explicit evidence and limitations.

### III. PROBLEM FORMULATION

#### A. Traffic Graph

Let the transportation sensing system be represented by a directed graph

$$G = (V, E, A), \quad (1)$$

where  $V$  is the set of traffic sensors or graph nodes,  $E$  is the set of directed connections, and  $A$  is the adjacency matrix. For METR-LA, the supplied graph contains 207 nodes, while PEMS-BAY contains 325 nodes.

The graph used by CPSTN is a benchmark traffic graph. It should not be interpreted as a complete geographic road graph because the supplied artifacts do not contain geographic coordinates for every segment or validated physical segment lengths.

#### B. Historical Traffic Tensor

At time  $t$ , let the historical observation matrix be

$$X_{t-T+1:t} = [X_{t-T+1}, \dots, X_t], \quad (2)$$

where each  $X_t$  contains one traffic-speed value per graph node. The supplied configuration uses

$$T = 12. \quad (3)$$

With a five-minute sampling cadence, twelve historical observations correspond to one hour of input history.

#### C. Forecasting Objective

The CPSTN model estimates twelve future observations:

$$\hat{Y}_{t+1:t+H} = f_x(X_{t-T+1:t}, G), \quad (4)$$

where

$$H = 12. \quad (5)$$

Because each forecast step corresponds to five minutes, the model produces forecasts at  $x = 5, 10, 15, \dots, 60$  minutes.

#### D. Dynamic Edge Cost

For a graph edge  $e = (u, v)$ , the prototype derives a deterministic proxy distance  $d_e$ . Let  $\hat{v}_e(t)$  denote the forecast speed associated with the edge at the selected forecast step and let  $v_{\min}$  be the operational lower bound.

The conceptual travel-time equation is

$$c_e(t) = \frac{d_e}{\max(\hat{v}_e(t), v_{\min})}. \quad (6)$$

The actual implementation performs the required unit conversion when combining proxy distance in kilometers and speed in miles per hour. The equation above describes the underlying relationship rather than suppressing the unit conversion.

#### E. Route Travel Time

For a candidate path  $P$ , the predicted route cost is

$$T(P, t) = \sum_{e \in P} c_e(t). \quad (7)$$

This value is a predicted travel-time proxy. It is not a measured travel time from a vehicle trajectory.

#### F. Route Savings

Let  $P_c$  be the current route and  $P_a$  an alternative route. Estimated savings are

$$\Delta T = T(P_c, t) - T(P_a, t). \quad (8)$$

The corresponding savings percentage is

$$S = \frac{\Delta T}{T(P_c, t)} \times 100. \quad (9)$$

#### G. Reliability Proxy

The prototype uses historical forecast RMSE as a proxy for forecast reliability. The proxy is not a probability and is not calibrated uncertainty. Let  $R$  denote the resulting score. The supplied policy requires

$$R \geq 70. \quad (10)$$

The score is derived from the supplied horizon-level RMSE and therefore represents historical forecast-error behavior rather than route-specific confidence.

#### H. Decision Rule

The supplied prototype applies the following policy:

$$\text{REROUTE} \iff (\Delta T \geq 2.0) \wedge (S \geq 8) \wedge (R \geq 70). \quad (11)$$

Otherwise,

$$\text{Decision} = \text{STAY}. \quad (12)$$

These thresholds are engineering policy parameters of the prototype. They are not presented as universally optimal transportation thresholds.

TABLE I  
CPSTN-DRAFT5 CONFIGURATION

| Parameter                        | Value     |
|----------------------------------|-----------|
| Input channels                   | 1         |
| Hidden dimension                 | 32        |
| Blocks                           | 3         |
| Dilations                        | [1, 2, 4] |
| Dropout                          | 0.1       |
| Input history                    | 12 steps  |
| Forecast horizon                 | 12 steps  |
| Cadence                          | 5 min     |
| Trainable parameters             | 26,668    |
| Total stored parameters/elements | 26,863    |
| Training seed                    | 42        |
| Batch size                       | 64        |
| Learning rate                    | 0.002     |
| Weight decay                     | 0.0001    |
| Patience                         | 12        |
| Gradient clipping                | 2.0       |

### IV. PROPOSED METHODOLOGY

#### A. System Architecture

The proposed prototype consists of two logically separate computational layers.

**Layer 1: Learned forecasting.** CPSTN-Draft5 receives historical traffic observations and the benchmark graph and predicts future traffic speed.

**Layer 2: Deterministic routing and decision.** Predicted speeds are transformed into dynamic edge costs. Candidate routes are generated, shortest-path search is performed, predicted travel times are compared, and the route policy determines whether a reroute recommendation should be returned.

The separation can be represented as

$$X_{\text{history}} \xrightarrow{\text{time}} \hat{V}_{\text{future}} \xrightarrow{\text{cost}} C_{\text{dynamic}} \xrightarrow{\text{routing}} \mathcal{P} \xrightarrow{\text{policy}} D. \quad (13)$$

Here  $\hat{V}_{\text{future}}$  is the predicted speed sequence,  $C_{\text{dynamic}}$  is the dynamic edge-cost representation,  $\mathcal{P}$  is the candidate-route set, and  $D$  is the final decision.

#### B. CPSTN-Draft5 Configuration

The supplied CPSTN-Draft5 model uses one input channel, hidden dimension 32, dropout 0.1, three blocks, and temporal dilations

$$[1, 2, 4]. \quad (14)$$

The model reports 26,668 trainable parameters and 26,863 total tensor elements including non-trainable stored parameters/buffers. The supplied training configuration uses seed 42, batch size 64, learning rate 0.002, weight decay  $10^{-4}$ , early-stopping patience 12, and gradient clipping 2.0.

#### C. Temporal Gating

Each temporal block uses a gated convolutional formulation. Let  $Z$  represent the block input. Two dilated temporal convolutions produce feature tensors  $A$  and  $B$ . The gated output can be expressed as

$$G(Z) = \tanh(A) \odot \sigma(B), \quad (15)$$

where  $\odot$  denotes element-wise multiplication and  $\sigma$  is the sigmoid function.

The use of multiple dilation factors allows the network to process temporal dependencies at multiple receptive-field scales. The supplied model uses three successive dilation factors of 1, 2, and 4.

#### D. Dense Graph Convolution

The graph component propagates information using the normalized adjacency matrix. Conceptually, for node feature representation  $H$  and normalized adjacency  $\tilde{A}$ ,

$$H_G = \tilde{A}HW_G, \quad (16)$$

where  $W_G$  represents a learned projection. In the supplied implementation, the operation is implemented through tensor reshaping, dense matrix multiplication, and a  $1 \times 1$  convolution.

The purpose is to allow node representations to incorporate information from graph-connected observations rather than relying only on temporal history at each node independently.

#### E. Residual Processing

The block contains temporal processing, graph propagation, feed-forward transformation, normalization/dropout, and residual combination. Residual connections provide a direct pathway between block input and transformed representation and allow multiple blocks to be stacked without requiring every transformation to learn a complete representation from scratch.

#### F. Input Normalization

The supplied system performs per-sensor z-score normalization:

$$x'_{i,t} = \frac{x_{i,t} - \mu_i}{\sigma_i}, \quad (17)$$

where  $\mu_i$  and  $\sigma_i$  are computed from the training period for sensor  $i$ .

The normalization parameters are dataset-specific. METR-LA and PEMS-BAY are therefore not assumed to share identical normalization statistics.

After inference, the forecast is inverse-transformed:

$$\hat{x}_{i,t} = \hat{x}'_{i,t}\sigma_i + \mu_i. \quad (18)$$

This is necessary for reporting predictions in the original speed scale.

#### G. Dataset Configuration

The supplied artifacts report the following dataset dimensions.

The graph dimensions are  $207 \times 207$  for METR-LA and  $325 \times 325$  for PEMS-BAY.

TABLE II  
DATASET CHARACTERISTICS

| Dataset  | Nodes | Train  | Validation | Test   |
|----------|-------|--------|------------|--------|
| METR-LA  | 207   | 23,967 | 3,416      | 6,844  |
| PEMS-BAY | 325   | 36,458 | 5,200      | 10,413 |

#### H. Dynamic Proxy Distance

The supplied benchmark adjacency provides graph connectivity but not physical segment length. The prototype therefore derives deterministic proxy distance from adjacency strength. This design is explicitly an engineering approximation.

This distinction is essential. A proxy distance does not imply that the resulting edge corresponds to a particular number of meters on a real road. Therefore, route distances and route travel times generated by this layer are internal benchmark-topology quantities.

#### I. Candidate Route Generation

The backend uses NetworkX weighted graph functionality. Candidate routes are generated under the current graph topology and ranked by dynamic cost. The implementation limits the candidate set to a configurable maximum, with four routes used by default.

The candidate generation procedure is deterministic for fixed input state and configuration. It does not learn route topology.

#### J. A\* Search

The routing implementation supports A\* search. Because the supplied benchmark graph has no geographic coordinates, a geographic lower-bound heuristic cannot be constructed. The prototype therefore uses

$$h(n) = 0. \quad (19)$$

For nonnegative edge costs, this is admissible and makes A\* equivalent to Dijkstra-style shortest-path search in terms of cost ordering.

This is a deliberate limitation rather than a missing feature. Introducing artificial coordinates solely to create a heuristic would create unsupported geographic information.

#### K. Route Cost Comparison

For each candidate route, the system computes predicted travel time using the selected forecast speed and proxy edge distances. The route with the lowest predicted travel time can then be compared with the currently selected route.

The decision layer deliberately uses both absolute and relative savings. Absolute savings prevent rerouting for negligible improvements when expressed as a percentage, while relative savings prevent decisions based solely on route scale.

---

**Algorithm 1** Dynamic Route Decision

---

**Require:** Current route  $P_c$ , candidate routes  $\mathcal{P}$ , forecast  $\hat{V}$   
**Ensure:** Decision  $D$

- 1: Compute current predicted travel time  $T(P_c)$
  - 2: Compute  $T(P)$  for each  $P \in \mathcal{P}$
  - 3: Select minimum-cost alternative  $P_a$
  - 4:  $\Delta T \leftarrow T(P_c) - T(P_a)$
  - 5:  $S \leftarrow 100\Delta T/T(P_c)$
  - 6: Compute reliability proxy  $R$
  - 7: **if**  $\Delta T \geq 2$  **and**  $S \geq 8$  **and**  $R \geq 70$  **then**
  - 8:      $D \leftarrow \text{REROUTE}$
  - 9: **else**
  - 10:     $D \leftarrow \text{STAY}$
  - 11: **end if**
  - 12: **return**  $D$
- 

#### L. Reliability Policy

The prototype derives a reliability score from historical forecast error. For example, the supplied five-minute METRLA RMSE is 3.9419. A normalized engineering score can be represented conceptually as

$$R = 100 \left( 1 - \frac{\text{RMSE}}{45} \right), \quad (20)$$

with clipping to the interval  $[0, 100]$ .

This transformation is not a calibrated statistical confidence interval. Its purpose is to provide a consistent scalar policy input derived from measured forecasting error.

#### M. Decision Layer

The decision procedure can be summarized by Algorithm 1.

#### N. Backend and Frontend Integration

The prototype exposes model and routing functionality through backend API services. The backend handles configuration loading, checkpoint loading, preprocessing, inference, route computation, comparison, diagnostics, and decision responses. The frontend presents the system state and route recommendation to the user.

The software architecture is intentionally modular. Model inference is separated from route generation, and route generation is separated from policy evaluation. This separation permits the forecasting model to be tested independently from the routing system and makes it possible to replace the proxy topology with a geographic graph in future work without redesigning the forecasting architecture.

#### O. Demo and Real Inference Modes

The prototype supports a demo workflow in which deterministic baseline history can be constructed from checkpoint sensor means. When the checkpoint is available, the resulting history is passed through the actual CPSTN model. This permits demonstration of the end-to-end software pipeline without claiming that the input represents live traffic.

The real inference interface requires a historical speed matrix with twelve time steps and the appropriate number of graph nodes. This is an important distinction between software capability and experimental evidence: accepting an input tensor through an API is not equivalent to experimentally demonstrating a live traffic feed.

### V. EXPERIMENTAL SETUP

#### A. Training Configuration

The supplied training configuration uses seed 42, batch size 64, learning rate 0.002, weight decay 0.0001, gradient clipping 2.0, and early-stopping patience 12. The supplied training summary reports early stopping after 63 epochs, with the best checkpoint at epoch 51 and a best validation loss of 0.1686676943.

The model uses twelve historical observations and predicts twelve future observations. At five-minute cadence, this corresponds to one hour of historical input and one hour of forecast output.

#### B. Evaluation Metrics

The supplied evaluation uses several complementary regression metrics.

Mean absolute error is

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|. \quad (21)$$

Mean squared error is

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2. \quad (22)$$

Root mean squared error is

$$\text{RMSE} = \sqrt{\text{MSE}}. \quad (23)$$

The coefficient of determination is

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}. \quad (24)$$

The supplied testing artifact also reports MAPE, sMAPE, WAPE, explained variance, and median absolute error. The MAPE implementation uses a denominator floor of 5.0.

#### C. Computational Environment

The supplied test artifact records the following environment:

- GPU: NVIDIA Tesla T4
- Python: 3.13.15
- PyTorch: 2.11.0+cu128
- CUDA: 12.8
- Reported inference time: approximately 0.01793 s for the reported test batch
- Reported throughput: approximately 3569.4 windows/s
- Peak GPU memory: approximately 2.139 GB

These values characterize the recorded experimental environment and should not be interpreted as universal runtime guarantees.

TABLE III  
OVERALL METR-LA TEST RESULTS

| Metric                | Result  |
|-----------------------|---------|
| MAE                   | 3.4454  |
| MSE                   | 47.3302 |
| RMSE                  | 6.8797  |
| MAPE                  | 9.5662% |
| sMAPE                 | 7.9628% |
| WAPE                  | 5.9622% |
| $R^2$                 | 0.74246 |
| Explained variance    | 0.74313 |
| Median absolute error | 1.50826 |
| Test windows          | 6,844   |

#### D. Experimental Honesty and Baselines

No independently executed competing forecasting baseline is included in the supplied artifacts. Consequently, this paper does not provide a model-comparison table claiming superiority over other traffic forecasting methods.

The ten supplied research papers are treated as literature context, not as experimental baselines. Their reported values, datasets, tasks, and evaluation protocols are not mixed with the CPSTN METR-LA results.

This separation is necessary because metrics from different tasks are not directly comparable. Road-extraction IoU, graph-topology scores, traffic-forecasting MAE, and route travel-time error measure different phenomena.

### VI. RESULTS

#### A. Overall METR-LA Forecasting Results

The supplied METR-LA testing artifact reports the results in Table III.

The results indicate that the model explains approximately 74.25% of the variance according to the supplied  $R^2$  value. The median absolute error is substantially lower than the mean absolute error, indicating that the error distribution contains a difference between its central tendency and the arithmetic average. No stronger statistical interpretation is made because the supplied artifacts do not provide a complete uncertainty or significance analysis.

#### B. Horizon-Wise Results

Table IV reports the supplied horizon-wise results.

The supplied experiment exhibits a consistent deterioration in point-forecast accuracy with increasing horizon. MAE rises from 2.2370 at five minutes to 4.3719 at sixty minutes. RMSE rises from 3.9419 to 8.5677. Meanwhile,  $R^2$  decreases from 0.91543 to 0.60064.

This pattern should be interpreted as an empirical characteristic of the evaluated METR-LA experiment. It should not be stated as a universal property of all traffic forecasting systems.

#### C. Implication for Routing

The horizon-wise results have a direct implication for the prototype routing layer. The first forecast step has the lowest supplied RMSE and therefore represents the most accurate

horizon in the evaluated experiment. The current route-decision implementation uses the first forecast step for predicted route-cost comparison.

This design avoids treating the entire sixty-minute forecast horizon as equally accurate. However, it also limits the routing layer's direct use of longer-horizon information. Future systems could use a multi-step route policy in which edge costs are integrated over time:

$$T(P) = \sum_{h=1}^H \sum_{e \in P} c_e(t+h), \quad (25)$$

but this would represent a methodological extension and is not claimed as part of the current prototype.

#### D. Routing and Decision Validation

The supplied validation artifacts test the complete route recommendation path using the benchmark graph and the real CPSTN checkpoint. Separate controlled tests exercise STAY and REROUTE decision behavior.

These tests provide evidence that the implemented components execute together. They do not establish that a recommended route is geographically correct, that its predicted travel time matches vehicle observations, or that users would experience measurable travel-time improvement.

This distinction is fundamental. A software integration test answers whether the implementation produces a route decision under controlled inputs. A field routing study would require geographic road geometry, actual travel distances, live or recorded traffic, vehicle trajectories, and a route-quality benchmark.

#### E. Computational Performance

The supplied test artifact reports approximately 0.01793 seconds for the reported inference batch and approximately 3569.4 windows per second in the recorded environment. Peak GPU memory is approximately 2.139 GB on a Tesla T4.

These measurements demonstrate that the supplied checkpoint can be executed efficiently in the recorded environment. They should not be interpreted as end-to-end user-perceived latency because network transfer, API serialization, route generation, frontend rendering, and live data acquisition are separate components.

### VII. DISCUSSION

#### A. Forecast Error and Temporal Horizon

The most direct experimental finding is the increase in point-prediction error with forecast horizon. At five minutes, RMSE is 3.9419 and  $R^2$  is 0.91543. At sixty minutes, RMSE is 8.5677 and  $R^2$  is 0.60064.

Traffic forecasting becomes more difficult as the prediction horizon increases because the model must project the evolution of a dynamic system farther into the future. In the present paper, however, the interpretation is restricted to the measured artifact. The supplied results demonstrate an empirical horizon effect; they do not establish its cause through a controlled ablation.

TABLE IV  
METR-LA HORIZON-WISE FORECASTING RESULTS

| Horizon (min) | MAE    | RMSE   | MAPE     | $R^2$   |
|---------------|--------|--------|----------|---------|
| 5             | 2.2370 | 3.9419 | 5.3992%  | 0.91543 |
| 10            | 2.5643 | 4.8326 | 6.4664%  | 0.87290 |
| 15            | 2.8160 | 5.4619 | 7.3170%  | 0.83765 |
| 20            | 3.0352 | 5.9739 | 8.0710%  | 0.80580 |
| 25            | 3.2382 | 6.4136 | 8.7858%  | 0.77616 |
| 30            | 3.4304 | 6.8077 | 9.4692%  | 0.74781 |
| 35            | 3.6103 | 7.1657 | 10.1224% | 0.72060 |
| 40            | 3.7802 | 7.4944 | 10.7430% | 0.69439 |
| 45            | 3.9378 | 7.7918 | 11.3145% | 0.66967 |
| 50            | 4.0898 | 8.0694 | 11.8604% | 0.64572 |
| 55            | 4.2337 | 8.3260 | 12.3724% | 0.62284 |
| 60            | 4.3719 | 8.5677 | 12.8731% | 0.60064 |

### B. Forecast Quality Versus Route Quality

Forecasting accuracy and routing quality are related but distinct.

Suppose two candidate routes have predicted travel times of 20 and 21 minutes. A forecast error of one or two minutes can change the route ordering. In contrast, if the predicted route times are 20 and 35 minutes, a comparable error may not change the decision.

This means that downstream route selection depends not only on absolute forecast error but also on the margin between competing routes. A future routing benchmark should therefore report route regret, travel-time difference, route-change frequency, and realized travel time rather than relying solely on forecasting MAE.

The present prototype does not provide such a controlled route-quality evaluation. It provides the computational mechanism needed to support one.

### C. Why the Reliability Proxy Is Limited

The reliability score is derived from historical forecast RMSE. This is useful as a transparent engineering signal, but it is not probabilistic confidence.

A calibrated uncertainty model would instead estimate a distribution such as

$$p(y_{t+h} \mid X_{t-T+1:t}, G), \quad (26)$$

or provide prediction intervals whose empirical coverage can be evaluated.

The current system does not provide this. Therefore, the term “reliability” is used with the explicit qualifier “proxy.”

### D. Threshold Sensitivity

The decision policy requires all three conditions:

$$\Delta T \geq 2, \quad S \geq 8, \quad R \geq 70. \quad (27)$$

This conjunctive rule is conservative in the sense that a route must satisfy every condition. However, the thresholds were not optimized against a route-level ground truth.

A future experiment could sweep the thresholds and evaluate the resulting trade-off between unnecessary rerouting and

missed beneficial rerouting. Such an experiment would require a route-level benchmark and realized travel times.

### E. Topology Versus Geography

The benchmark graph is suitable for studying spatio-temporal traffic forecasting, but it does not automatically become a geographic navigation graph.

The prototype’s proxy-distance construction provides deterministic edge weights so that travel-time calculations can be demonstrated. This is useful for software integration and reproducibility. Nevertheless, proxy distance cannot substitute for road length when making claims about real-world travel.

A geographic deployment would require at least:

- 1) geographic coordinates for nodes or road vertices,
- 2) physical road-segment lengths,
- 3) directionality and turn restrictions,
- 4) road-class and speed-limit information where relevant,
- 5) validated map topology,
- 6) real-time or recorded traffic observations,
- 7) GPS trajectory data for route evaluation.

None of these requirements should be assumed to exist merely because the prototype exposes a route API.

### F. Generalization

The supplied project includes both METR-LA and PEMS-BAY configurations. Their node counts and graph structures differ. The existence of a second dataset configuration is useful for reproducibility, but it does not by itself establish external geographic generalization.

A rigorous generalization study would train or evaluate under controlled cross-city settings and report the resulting degradation. The current evidence does not support such a claim.

## VIII. LIMITATIONS AND THREATS TO VALIDITY

### A. Benchmark Graph Limitation

The largest limitation is the distinction between benchmark topology and geographic road topology. The graph is appropriate for traffic forecasting but does not supply all information required for navigation.

### B. Proxy-Distance Limitation

The routing layer requires edge distance. Because physical road lengths are unavailable, the prototype derives deterministic proxy distances from adjacency strength. These distances are suitable for a reproducible computational demonstration but should not be interpreted as measured road lengths.

### C. Live Traffic Limitation

The supplied artifacts do not establish an experimentally validated live traffic ingestion pipeline. Demo mode uses deterministic baseline histories. Consequently, the paper does not claim that the system has been validated against continuously changing real-world traffic feeds.

### D. GPS Limitation

The supplied benchmark artifacts do not provide a validated geographic coordinate system for the full routing graph. The current study therefore does not demonstrate real-time user-location routing.

A future integration could accept a GPS position and map-match it to a geographic graph, but that capability belongs to future work unless independently supplied and experimentally validated.

### E. Decision-Threshold Limitation

The thresholds of two minutes, eight percent, and reliability 70 are prototype policy parameters. They are not learned from a route-level optimization objective, nor are they established as universal transportation thresholds.

### F. No Controlled Forecasting Baseline

Because no independently executed competing baseline is supplied, the paper does not claim state-of-the-art performance or superiority.

### G. No Route-Level Ground Truth

The system demonstrates route computation and policy execution but lacks a controlled benchmark of realized travel times for the generated alternatives. Therefore, the study cannot establish route regret or actual travel-time savings.

### H. Frontend Environment Limitation

The supplied validation report indicates that frontend build validation was blocked by unavailable npm registry access in the preparation environment. The frontend source and integration are therefore distinguished from a claim of successful clean-room installation under every environment.

### I. Dataset and Domain Limitations

METR-LA and PEMS-BAY are benchmark traffic datasets rather than universal representations of transportation networks. Dataset-specific sensor distributions, graph structure, sampling patterns, and traffic regimes can affect model behavior.

TABLE V  
REPRODUCIBILITY CONFIGURATION

| Item             | Supplied value     |
|------------------|--------------------|
| Model            | CPSTN-Draft5       |
| History          | 12 steps           |
| Horizon          | 12 steps           |
| Cadence          | 5 minutes          |
| Seed             | 42                 |
| Batch            | 64                 |
| Learning rate    | 0.002              |
| Weight decay     | 0.0001             |
| Patience         | 12                 |
| Clip norm        | 2.0                |
| Hidden dimension | 32                 |
| Blocks           | 3                  |
| Dilations        | 1, 2, 4            |
| Dropout          | 0.1                |
| Normalization    | per-sensor z-score |

## IX. REPRODUCIBILITY

The supplied artifacts contain the major configuration information required to reproduce the reported forecasting experiment, including model architecture, dataset dimensions, input history, forecast horizon, normalization strategy, optimizer settings, seed, checkpoint information, metrics, and recorded hardware/software environment.

The key configuration is summarized below:

Raw benchmark files are not duplicated in the supplied prototype package. Reproduction therefore requires access to the appropriate benchmark datasets and their graph/normalization metadata.

The checkpoint should be evaluated using the same dataset-specific normalization and graph artifacts. Mixing graph or normalization metadata between METR-LA and PEMS-BAY would invalidate the intended evaluation procedure.

## X. FUTURE WORK

The present prototype provides a foundation for several experimentally testable extensions.

### A. Geographic Graph Integration

The benchmark topology can be replaced or augmented by a geographic road graph containing coordinates and physical segment lengths. This would permit geographically meaningful A\* heuristics and route visualization.

### B. Real-Time User Location

A future system can map-match a user's GPS location to the geographic road graph and dynamically identify the current route and nearby alternatives. Such a capability requires a geographic graph and GPS validation and is not claimed as part of the current experiment.

### C. Live Traffic Integration

A validated traffic-data source can replace deterministic demo history. The system could periodically ingest speed observations, update the twelve-step history window and refresh route costs.

#### D. Uncertainty Calibration

Instead of the current RMSE-based reliability proxy, future work can estimate predictive uncertainty and evaluate calibration. Metrics such as interval coverage, calibration error, and negative log likelihood could complement point-forecast metrics.

#### E. Route-Quality Benchmarking

A route-level benchmark should compare predicted routes against realized travel times. Candidate metrics include:

$$\text{Regret} = T(P_{\text{selected}}) - T(P_{\text{oracle}}), \quad (28)$$

where the oracle route is defined using realized traffic conditions.

Additional measures can include reroute frequency, realized savings, missed opportunities, unnecessary reroutes, and route stability.

#### F. Ablation Studies

Ablation experiments could remove graph convolution, temporal gating, individual dilation scales, or the decision reliability criterion. This would help determine which architectural and policy components materially affect downstream behavior.

#### G. Threshold Optimization

The current thresholds can be tuned using a validation objective based on route regret and rerouting cost. Such an experiment would replace engineering defaults with empirically calibrated policy values.

#### H. External-City Validation

External-city evaluation can test whether the forecasting and routing integration transfers beyond the supplied datasets. This should be conducted without mixing training normalization statistics or graph structures between datasets.

#### I. User-Level Evaluation

A field evaluation could compare route recommendations with observed trajectories and measure actual travel-time impact. Such an experiment would be necessary before making claims about user-level navigation benefit.

### XI. CONCLUSION

This paper presented a CPSTN-centered framework that integrates spatio-temporal traffic-speed forecasting with deterministic dynamic route recommendation over benchmark traffic graphs. CPSTN-Draft5 uses temporal gating and dense graph convolution to process twelve historical observations and predict twelve future speed steps at five-minute intervals. The downstream routing layer converts predicted speed into dynamic travel-time costs, generates candidate paths, performs shortest-path search using a zero heuristic, compares route costs, computes a historical forecast-error proxy, and applies a configurable REROUTE/STAY decision.

On the supplied METR-LA test artifact, CPSTN-Draft5 reports MAE 3.4454, RMSE 6.8797, MAPE 9.5662%, sMAPE

7.9628%, WAPE 5.9622%, and  $R^2$  0.74246 across 6,844 test windows. Horizon-wise performance is strongest at five minutes and degrades toward sixty minutes in the supplied experiment. The implementation demonstrates that forecast output can be connected to a deterministic route-decision layer without treating  $A^*$  as a learned component.

The principal qualification is the topology of the supplied benchmark. Because geographic coordinates and physical road lengths are not available, the routing layer uses deterministic proxy distances. The resulting route computations are therefore benchmark-topology demonstrations rather than validated geographic navigation. Likewise, the supplied evidence does not establish live traffic routing, real-time GPS navigation, calibrated probabilistic reliability, or actual user travel-time savings.

The work provides a reproducible integration architecture and a clear basis for future experiments. Geographic road integration, live traffic ingestion, uncertainty calibration, route-quality benchmarks, ablation studies, external-city validation, and user-level evaluation are the necessary next steps for moving from a benchmark-oriented prototype toward a rigorously validated navigation system.

### REFERENCES

- [1] A. Batra, S. Singh, G. Pang, S. Basu, C. V. Jawahar, and M. Paluri, "Improved Road Connectivity by Joint Learning of Orientation and Segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 10385–10393, doi: 10.1109/CVPR.2019.01063.
- [2] F. Bastani and S. Madden, "Beyond Road Extraction: A Dataset for Map Update Using Aerial Images," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021, pp. 11905–11914.
- [3] A. Van Etten, "City-Scale Road Extraction from Satellite Imagery v2: Road Speeds and Travel Times," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, 2020, pp. 1786–1795.
- [4] J. D. Wegner, J. A. Montoya-Zegarza, and K. Schindler, "A Higher-Order CRF Model for Road Network Extraction," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2013, pp. 1698–1705, doi: 10.1109/CVPR.2013.222.
- [5] D. Belli and T. Kipf, "Image-Conditioned Graph Generation for Road Network Extraction," arXiv:1910.14388, 2019. Presented in the NeurIPS 2019 Workshop on Graph Representation Learning.
- [6] Y. Xu, Z. Xie, Y. Feng, and Z. Chen, "Road Extraction from High-Resolution Remote Sensing Imagery Using Deep Learning," *Remote Sens.*, vol. 10, no. 9, Art. no. 1461, 2018, doi: 10.3390/rs10091461.
- [7] L. Duan, W. Mapurisa, M. Leras, L. Lotter, and Y. Tarabalka, "BrightEarth Roads: Towards Fully Automatic Road Network Extraction from Satellite Imagery," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, 2024, pp. 9192–9196, doi: 10.1109/IGARSS53475.2024.10641133.
- [8] H. Sui, N. Zhou, M. Zhou, and L. Ge, "Vector Road Map Updating from High-Resolution Remote-Sensing Images with the Guidance of Road Intersection Change Detection and Directed Road Tracing," *Remote Sens.*, vol. 15, no. 7, Art. no. 1840, 2023, doi: 10.3390/rs15071840.
- [9] Q. Zhang and I. Couloigner, "A Framework for Road Change Detection and Map Updating," in *Int. Archives Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. XXXV, Part B2, 2004, pp. 729–734.
- [10] Q. Zhu, Y. Zhang, L. Wang, Y. Zhong, Q. Guan, X. Lu, L. Zhang, and D. Li, "A Global Context-aware and Batch-independent Network for Road Extraction from VHR Satellite Imagery," *ISPRS J. Photogramm. Remote Sens.*, vol. 175, pp. 353–365, 2021, doi: 10.1016/j.isprsjprs.2021.03.016.